* Alguns dos Exercícios foram retirados/adaptados do curso Python Programming MOOC 2026

1) Classe **Livro**: Crie e teste uma classe que modele um livro:

   Atributos: `título, autor, total_paginas, pagina_atual`
   Métodos: `avancar_pagina, voltar_pagina, ir_para_pagina, __str__`

   Obs: O método `avancar_pagina` deve incrementar a página atual em 1, e o `voltar_pagina` deve diminuir em 1 (sem permitir valores negativos). O método `ir_para_pagina` recebe um número e define a página atual, respeitando os limites do livro.

2) Classe Cronômetro **(Stopwatch)**:

   Considere a seguinte estrutura para a classe Stopwatch:

```python
class Stopwatch:
    def __init__(self):
        self.seconds = 0
        self.minutes = 0
```

   Adicione funcionalidades à definição da classe para que ela funcione da seguinte forma:

```python
watch = Stopwatch()
for i in range(3600):
    print(watch)
    watch.tick()
```

   Exemplo de Saída:

```
00:00
00:01
00:02
... muitas outras linhas impressas
00:59
01:00
01:01
... muitas, muitas outras linhas impressas
59:58
59:59
00:00
00:01
```

   O método `tick` adiciona um segundo ao cronômetro. O valor máximo tanto para os segundos quanto para os minutos é 59. Sua definição de classe também deve conter um método `__str__`, que retorna uma representação em string do estado do cronômetro, como mostrado no exemplo acima.

3) Classe Relógio **(Clock)**:

Defina uma nova classe chamada Clock que expanda as capacidades da sua classe Stopwatch. Ela deve funcionar da seguinte forma:

```
clock = Clock(23, 59, 55)
print(clock)
clock.tick()
print(clock)
clock.tick()
print(clock)
clock.tick()
print(clock)
clock.tick()
print(clock)
clock.tick()
print(clock)
clock.tick()
print(clock)
clock.set(12, 5)
print(clock)
```

Exemplo de Saída:

```
23:59:55
23:59:56
23:59:57
23:59:58
23:59:59
00:00:00
00:00:01
12:05:00
```

O construtor deve receber valores iniciais para horas, minutos e segundos como argumentos e defini-los adequadamente. O método `tick` adiciona um segundo ao relógio. O método set define novos valores para as horas e minutos, e define os segundos como zero.

Em um segundo momento de teste, ajuste seu relógio para a hora atua e crie em sua função principal um laço que faça seu relógio atualizar a cada segundo automaticamente.

4) Cartão de Almoço (**LunchCard**)

Parte1: Crie uma nova classe chamada `LunchCard`. Escreva o construtor (recebe o saldo inicial como argumento) e o método `__str__`, que retorna a string: "The balance is X euros" com precisão de uma casa decimal.

```
card = LunchCard(50)
print(card)
# Saída esperada: The balance is 50.0 euros
```

Parte 2: Implemente os métodos:
- `eat_lunch`: subtrai 2.60 euros do saldo.
- `eat_special`: subtrai 4.60 euros do saldo.

O saldo nunca deve ficar negativo. Se não houver dinheiro suficiente, o valor não deve ser subtraído.

Parte 3: Implemente o método `deposit_money(amount)` que aumenta o saldo. Se o valor passado for menor que zero, deve lançar uma exceção do tipo `ValueError`.

Parte 4: Escreva a função principal executando a seguinte sequência:

- Crie um cartão para o Peter (saldo inicial de 20 euros).
- Crie um cartão para a Grace (saldo inicial de 30 euros)
- Peter come um almoço especial.
- Grace come um almoço regular.
- Imprima o saldo de cada cartão (linhas separadas, com o nome no início).
- Peter deposita 20 euros.
- Grace come o almoço especial.
- Imprima o saldo de cada cartão.
- Peter come um almoço regular duas vezes seguidas.
- Grace deposita 50 euros.
- Imprima o saldo de cada cartão.

5) Cartão de Almoço e Terminal de Pagamento **(LunchCard e PaymentTerminal)**

Na parte anterior, houve um exercício onde você implementou a classe `LunchCard`. O cartão possuía métodos separados para comer um almoço regular e um especial, além de um método para depositar dinheiro no cartão.

A classe `LunchCard`, da forma como foi solicitada, possui alguns problemas. O próprio cartão tinha conhecimento dos preços das diferentes opções de almoço e sabia subtrair o valor correto do saldo com base nisso. Mas imagine que os preços mudassem ou novos itens fossem introduzidos no sistema, enquanto vários cartões já estivessem cadastrados. Isso significaria que todos os cartões existentes precisariam ser substituídos por versões atualizadas com os novos preços.

Uma solução melhor seria tornar os cartões "burros", sem conhecimento sobre os preços dos produtos. O único propósito do cartão seria simplesmente manter o controle do saldo disponível. Todas as funcionalidades mais complexas devem ficar contidas em outra classe: o terminal de pagamento.

Parte 1: Um LunchCard mais simples

Vamos primeiro implementar uma versão mais simples da classe `LunchCard`. O cartão deve conter apenas a funcionalidade de consultar o saldo atual, depositar dinheiro no cartão e subtrair do saldo. Preencha o método `subtract_from_balance(amount)` no modelo do exercício de acordo com as instruções nos comentários:

```python
class LunchCard:
    def __init__(self, balance: float):
        self.balance = balance

    def deposit_money(self, amount: float):
        self.balance += amount

    def subtract_from_balance(self, amount: float) -> bool:
        pass
        # O valor deve ser subtraído do saldo apenas se
        # houver dinheiro suficiente no cartão.
        # Se o pagamento for bem-sucedido, o método retorna True.
        # Caso contrário, retorna False.
```

Você pode usar o seguinte código para testar sua função:

```python
if __name__ == "__main__":
    card = LunchCard(10)
    print("Balance", card.balance)
    result = card.subtract_from_balance(8)
    print("Payment successful:", result)
    print("Balance", card.balance)
    result = card.subtract_from_balance(4)
    print("Payment successful:", result)
    print("Balance", card.balance)
```

Exemplo de saída:

```
Balance 10
Payment successful: True
Balance 2
Payment successful: False
Balance 2
```

Parte 2: O terminal de pagamento e pagamentos em dinheiro

No restaurante universitário é possível pagar em dinheiro ou com o `LunchCard`. Um terminal de pagamento é usado para processar transações tanto em dinheiro quanto no cartão. Vamos começar com as transações em dinheiro.

Abaixo temos a estrutura inicial para a classe `PaymentTerminal`. Implemente os métodos conforme descrito nos comentários:

```python
class PaymentTerminal:
    def __init__(self):
        # Inicialmente há 1000 euros em dinheiro disponíveis no terminal
        self.funds = 1000
        self.lunches = 0
        self.specials = 0

    def eat_lunch(self, payment: float) -> float:
```

```
        # Um almoço regular custa 2.50 euros.
        # Aumente o valor dos fundos no terminal pelo
        # preço do almoço, incremente o número de almoços vendidos
        # e retorne o troco adequado.
            # Se o pagamento passado como argumento não for grande o
suficiente para cobrir
        # o preço, o almoço não é vendido e o valor integral é retornado.
        pass

    def eat_special(self, payment: float) -> float:
        # Um almoço especial custa 4.30 euros.
        # Aumente o valor dos fundos no terminal pelo
        # preço do almoço, incremente o número de almoços vendidos
        # e retorne o troco adequado.
            # Se o pagamento passado como argumento não for grande o
suficiente para cobrir
        # o preço, o almoço não é vendido e o valor integral é retornado.
        pass
```

Você pode usar o seguinte código para testar sua classe:

```
exactum = PaymentTerminal()
change = exactum.eat_lunch(10)
print("The change returned was", change)
change = exactum.eat_lunch(5)
print("The change returned was", change)
change = exactum.eat_special(4.3)
print("The change returned was", change)
print("Funds available at the terminal:", exactum.funds)
print("Regular lunches sold:", exactum.lunches)
print("Special lunches sold:", exactum.specials)
```

Exemplo de saída:

```
The change returned was 7.5
The change returned was 2.5
The change returned was 0.0
Funds available at the terminal: 1009.3
Regular lunches sold: 2
Special lunches sold: 1
```

Parte 3: Pagamentos com cartão

Agora vamos implementar as transações com cartão. Precisaremos de métodos que recebam um `LunchCard` como argumento e reduzam o saldo do cartão pelo preço do almoço:

```
class PaymentTerminal:
    # ...
```

```
    def eat_lunch_lunchcard(self, card: LunchCard) -> bool:
        # Um almoço regular custa 2.50 euros.
        # Se houver dinheiro suficiente no cartão,
        # subtraia o preço do almoço do saldo
        # e retorne True. Se não, retorne False.
        pass

    def eat_special_lunchcard(self, card: LunchCard) -> bool:
        # Um almoço especial custa 4.30 euros.
        # Se houver dinheiro suficiente no cartão,
        # subtraia o preço do almoço do saldo
        # e retorne True. Se não, retorne False.
        pass
```

Nota: ao pagar com um `LunchCard`, os fundos em dinheiro disponíveis no terminal não mudam. No entanto, os almoços ainda são vendidos sempre que houver o saldo necessário, portanto lembre-se de incrementar o número de almoços vendidos adequadamente.

Você pode usar o seguinte código para testar sua classe:

```
exactum = PaymentTerminal()

change = exactum.eat_lunch(10)
print("The change returned was", change)

card = LunchCard(7)

result = exactum.eat_special_lunchcard(card)
print("Payment successful:", result)
result = exactum.eat_special_lunchcard(card)
print("Payment successful:", result)
result = exactum.eat_lunch_lunchcard(card)
print("Payment successful:", result)

print("Funds available at the terminal:", exactum.funds)
print("Regular lunches sold:", exactum.lunches)
print("Special lunches sold:", exactum.specials)
```

Exemplo de saída:

```
The change returned was 7.5
Payment successful: True
Payment successful: False
Payment successful: True
Funds available at the terminal: 1002.5
Regular lunches sold: 1
Special lunches sold: 1
```

Parte 4: Recarregando o cartão

Para finalizar, adicione um método para recarregar o cartão na classe `PaymentTerminal`:

```
def deposit_money_on_card(self, card: LunchCard, sum: float):
    # O valor depositado deve ser adicionado ao saldo do cartão
    # e também somado aos fundos do terminal de pagamento.
     # Se o valor a ser depositado for negativo, a operação não deve ser
realizada.
    pass
```

Você pode usar o seguinte código para testar seu método:

```
exactum = PaymentTerminal()

card = LunchCard(2)
print(f"Card balance is {card.balance} euros")

result = exactum.eat_special_lunchcard(card)
print("Payment successful:", result)

exactum.deposit_money_on_card(card, 100)
print(f"Card balance is {card.balance} euros")

result = exactum.eat_special_lunchcard(card)
print("Payment successful:", result)
print(f"Card balance is {card.balance} euros")

print("Funds available at the terminal:", exactum.funds)
print("Regular lunches sold:", exactum.lunches)
print("Special lunches sold:", exactum.specials)
```

Exemplo de saída:

```
Card balance is 2 euros
Payment successful: False
Card balance is 102 euros
Payment successful: True
Card balance is 97.7 euros
Funds available at the terminal: 1100.0
Regular lunches sold: 0
Special lunches sold: 1
```

6) **Classe Personagem:** Criando um Sistema de Batalha

Você foi contratado para desenvolver um sistema básico de batalha para um jogo de RPG. Para isso, implemente uma classe chamada `Personagem`, que deve conter os seguintes atributos e métodos:

Atributos:

- `nome` (string): Nome do personagem.
- `vida` (int, padrão 100): Representa os pontos de vida do personagem.
- `ataque` (int, padrão 10): Indica o dano causado ao atacar.
- `nivel` (int, padrão 1): O nível do personagem.

Métodos:

- `atacar(inimigo: "Personagem")`: O personagem ataca o `inimigo`, chamando o método `receber_dano` dele com o valor do seu próprio ataque. Exibe uma mensagem informando o ataque.
- `receber_dano(dano: int)`: Reduz os pontos de vida do personagem pelo valor do dano, exibe a vida restante e, caso a vida chegue a 0 (ou menos), exibe que o personagem foi derrotado.
- `subir_nivel()`: Aumenta o nível em +1, incrementa +5 no ataque e +20 na vida atual. Exibe a mensagem de evolução.
- `__str__()`: Imprime uma string formatada com o nome, nível, vida e ataque do personagem.

Teste:

1. Crie dois personagens chamados **"Rockeiro"** e **"Pirangueiro"**:
    - O **Rockeiro** começa com `ataque = 12` e `vida = 120`.
    - O **Pirangueiro** começa com `ataque = 15` e `vida = 150`.
2. Faça o Rockeiro atacar o Pirangueiro.
3. Faça o Pirangueiro atacar o Rockeiro.
4. Suba o nível do Rockeiro.
5. Exiba o status do Rockeiro.

Saída Esperada:

```
Rockeiro atacou Pirangueiro causando 12 de dano!
Pirangueiro agora tem 138 de vida.
Pirangueiro atacou Rockeiro causando 15 de dano!
Rockeiro agora tem 105 de vida.
Rockeiro subiu para o nível 2! Ataque aumentado para 17, Vida agora é 125.
Nome: Rockeiro | Nível: 2 | Vida: 125 | Ataque: 17
```